

Design: Recording Finalization Reliability

Design specification for sealing active and background recordings when the live KSP vessel ends before Parsek can run the normal scene-exit finalizer. This document is the contract for future implementation PRs; it does not describe shipped behavior yet.

Related docs: [parsek-flight-recorder-design.md](#) , [parsek-rewind-to-separation-design.md](#) , [dev/plans/incomplete-ballistic-extrapolation.md](#) , [dev/plans/recording-finalization-reliability.md](#) .

Problem

Parsek can synthesize a predicted tail for some incomplete recordings when the player exits the flight scene while the live vessel still exists. That is not enough for mission-tree correctness. A recording can end because the player switches focus, a background sibling leaves the physics bubble, KSP deletes an unfocused atmospheric vessel, the active vessel crashes before scene exit, or `FlightGlobals` is unavailable during teardown. In those cases Parsek may commit only the last sampled point, infer a weak terminal state from stale data, or mark the vessel destroyed without the synthetic trajectory that explains how it got there.

This is foundational for Rewind to Separation. The Unfinished Flights classifier needs trustworthy terminal state and end-time data to decide whether a sibling really ended badly and whether a rewind point should stay actionable. A terminal state that depends on which KSP object happened to survive until scene exit is not a durable gameplay contract.

Current Implementation Audit

The current implementation has useful pieces:

- `IncompleteBallisticSceneExitFinalizer.TryApply` snapshots a live vessel's patched-conic chain, then runs `BallisticExtrapolator.Extrapolate` when needed.
- `BallisticExtrapolator.ShouldExtrapolate` correctly targets flying, suborbital, escaping, and low-periapsis orbital cases while skipping landed, splashed, prelaunch, docked, and stable orbiting vessels.
- `BackgroundRecorder` records loaded physics-bubble siblings, on-rails orbit checkpoints, part death and joint-break events, debris TTL expiry, and background-vessel destruction.
- `FinalizeTreeRecordings` runs for scene exit, manual commit, revert commit, and post-destruction merge paths.

The reliability gaps are about timing and ownership:

End mode	Current behavior	Gap
Scene exit, live vessel exists	Scene-exit finalizer can append predicted/extrapolated segments and terminal state.	Works only if the vessel and <code>FlightGlobals</code> are still usable at that moment.
Scene exit, vessel missing	Falls back to trajectory inference or prior destroyed state.	No last-known predicted tail exists after the vessel object is gone.
Active crash before scene exit	<code>FlightRecorder.OnVesselWillDestroy</code> samples final position and marks destroyed; post-destruction commit runs with <code>isSceneExit:false</code> .	No ballistic/predicted tail is synthesized for the last live approach to impact.

Background vessel destroyed	<code>BackgroundRecorder.OnBackgroundVesselWillDestroy</code> flushes track sections and persists the sidecar.	No finalization tail; terminal data depends on whatever was already sampled.
Debris TTL or out-of-bubble end	<code>EndDebrisRecording</code> sets <code>ExplicitEndUT</code> and either destroyed or situation-derived terminal state.	No predicted continuation to the KSP deletion/destruction endpoint.
Focus switch / backgrounding	The active recorder transitions to background or promotes another tree member.	A recording that later becomes unreachable still depends on scene-exit or destruction-time inference.
UI maneuver node present	Prior code recorded a segment ending in <code>Maneuver</code> , marked <code>StoppedBeforeManeuver</code> , and treated that as an orbiting terminus; Phase 2 retires that terminal behavior by falling back to current-state propagation when a UI node is detected.	Planned UI burns are hypothetical and must not truncate the real projected coast.

The design target is to keep the existing finalizer and extrapolator, but make their output available before the live vessel disappears.

Terminology

- **Finalization** - sealing a `Recording` with a terminal state, authoritative `ExplicitEndUT`, terminal orbit/surface metadata, and any synthetic predicted tail needed for playback.
- **Synthetic tail** - predicted or extrapolated trajectory data after the last real recorded sample. It is playback/terminal metadata, not a replacement for actual history.
- **Finalization cache** - an in-memory, last-known-good synthetic tail for one live recording owner. It is refreshed while the vessel exists and consumed when the vessel ends unexpectedly.
- **Active recorder cache** - cache owned by the focused `FlightRecorder`.
- **Background cache** - cache owned by `BackgroundRecorder` per tracked background PID/recording id.
- **Premature end** - any recording-end path where Parsek cannot rely on a fresh live-vessel scene-exit finalizer: crash, KSP delete, physics-bubble unload, debris TTL expiry, focus-switch abandonment, or teardown with unavailable runtime APIs.
- **Actual projected chain** - the future path implied by the vessel's current state vector and current thrust/coast state. It explicitly excludes maneuver nodes and other UI planning tools.

Mental Model

Recording has two independent products:

```
actual history, sampled as the player flies
-> trajectory points, orbit checkpoints, part events

latest projected ending, refreshed while the vessel exists
-> predicted/extrapolated tail, terminal UT, terminal state
```

Only the first product describes what the vessel already did. The second product answers: "If Parsek loses this vessel now, what would KSP consider the natural end from this point?"

The cache is a safety net:

```
while recording:
    sample real history normally
    every 5 seconds, or on a meaningful state event:
        refresh the finalization cache from the vessel's current state
        (live or on-rails)

when a recording ends:
    if live vessel is still valid:
        compute one fresh finalization result and apply it
    else if a cache exists:
        trim the cached tail to the recording's last real sample UT and apply it
    else:
        use today's inference fallback and log the degraded result
```

KSP's unload model gives the policy its shape. Unloaded vessels move on rails; atmospheric forces are not applied to distant vessels. Community-facing KSP API docs describe unloaded/on-rails motion as orbital-only rather than full-atmosphere physics, and KSP players document the stock auto-delete behavior for objects below the body-specific deletion altitude and outside the active vessel's physics bubble. Parsek should mirror that gameplay outcome: in-atmosphere unfocused deletion is `Destroyed` ; stable vacuum orbit is `Orbiting` .

Gameplay Scenarios

Booster Separation in Atmosphere

The player launches a two-stage rocket. The first stage has a probe core and parachutes, so it becomes a controllable sibling at staging. The player continues flying the upper stage. The booster falls behind, leaves the loaded physics bubble, and stock KSP deletes it once it is outside range and still below the atmospheric deletion threshold.

Expected Parsek behavior:

- The booster background recording has its own finalization cache.
- While the booster is loaded, the cache refreshes from its falling state.
- When KSP unloads/deletes the booster, Parsek consumes the cache, appends the predicted/descent tail, sets `TerminalStateValue = Destroyed` , and records the terminal UT.
- The booster appears as an Unfinished Flight if its parent branch has a Rewind Point.

Mun Lander and Orbiter

The player undocks a Mun lander from an orbiter. The lander is active; the orbiter is a background recording in a stable Mun orbit. The player lands and exits to the Space Center.

Expected Parsek behavior:

- The lander finalizes from a fresh live vessel on scene exit.
- The orbiter background cache says the orbiter remains in stable orbit.
- If the orbiter is still live, finalization can refresh it; if not, the cache seals it as `Orbiting` .
- No synthetic atmospheric destruction is invented for a stable vacuum orbit.

Scene Exit Mid-Burn

The player clicks Space Center during ascent while engines are firing.

Expected Parsek behavior:

- The cache may be up to 5 seconds stale, but the scene-exit path should still make one fresh live-vessel finalization attempt before teardown.
- If the live attempt succeeds, it wins over the cache.
- If KSP has already torn down the vessel/runtime, the cache is good enough to avoid a frozen mid-burn endpoint.

Focused Crash

A re-entry craft hits terrain while focused. `onVesselWillDestroy` fires, the part-death cascade begins, and a post-destruction tree merge may happen before scene exit.

Expected Parsek behavior:

- The last pre-impact cache already predicts destruction near the crash UT.
- The crash/coalescer finish path consumes that cache instead of relying only on the final sampled point.
- The recording gets `Destroyed`, a terminal UT, and a short tail that carries playback into the impact/destruction endpoint.

Vessel Switch Abandonment

The player switches away from a controllable sibling and never returns to it. It backgrounds, goes on rails, then later unloads or is destroyed by stock rules.

Expected Parsek behavior:

- Switching focus alone is not a terminal event.
- The recording owner changes from active to background, but the cache follows the recording id.
- When the background vessel actually ends or becomes unreachable, Parsek consumes that recording's cache.

Rewind to Separation Dependency

A staged booster ended in the background and the player wants to re-fly it. The UI must know whether the booster is genuinely crashed/destroyed, still orbiting, safely landed, or unfinalized.

Expected Parsek behavior:

- `TerminalKindClassifier` receives a reliable terminal state instead of stale last-sample inference.
- Unfinished Flights membership is based on a committed finalization contract, not on scene-exit timing.

Data Model

The cache is in-memory only. It is not serialized, and it does not change the recording sidecar format. Recordings are still committed only when a recording ends.

Suggested shape:

```
internal sealed class RecordingFinalizationCache
{
    public string RecordingId;
    public uint VesselPersistentId;
    public FinalizationCacheOwner Owner; // ActiveRecorder, BackgroundLoaded,
```

```

BackgroundOnRails
    public FinalizationCacheStatus Status;    // Empty, Fresh, Stale, Failed

    public double CachedAtUT;
    public float CachedAtRealtime;
    public string RefreshReason;
    public string DeclineReason;

    public double LastObservedUT;
    public string LastObservedBodyName;
    public Vessel.Situations LastSituation;
    public bool LastWasInAtmosphere;
    public bool LastHadMeaningfulThrust;

    public double TailStartsAtUT;
    public double TerminalUT;
    public TerminalState? TerminalState;
    public string TerminalBodyName;
    public RecordingFinalizationTerminalOrbit? TerminalOrbit;
    public SurfacePosition? TerminalPosition;
    public double? TerrainHeightAtEnd;

    public List<OrbitSegment> PredictedSegments; // isPredicted = true
}

```

Ownership:

- `FlightRecorder` owns one cache for the focused recording it is currently sampling.
- `BackgroundRecorder` owns a cache per `BackgroundMap` entry or per loaded/on-rails state. The cache is keyed by recording id and vessel PID so it survives active -> background and background -> active promotion.
- `VesselPersistentId = 0` means unknown legacy/test identity. Cache application still requires `RecordingId` to match, and when both sides have a nonzero vessel PID, those PIDs must match too.
- The cache payload is copied into the target `Recording` only when a consumer finalizes that recording.

No `Recording` field is needed for the cache itself. Applying a cache reuses existing persisted fields:

`OrbitSegments` , `ExplicitEndUT` , `TerminalStateValue` , terminal orbit fields, `TerminalPosition` , and `TerrainHeightAtEnd` .

Behavior

Refresh Policy

Refresh every 5 seconds while a recording has a live vessel reference, plus forced refreshes on meaningful state changes:

- recording start or promotion from background
- active -> background transition
- throttle crossing idle/active thresholds
- engine or RCS activity while loaded
- atmosphere entry or exit
- SOI change

- go-on-rails or go-off-rails
- vessel situation change
- staging, undock, EVA, joint break, and part death
- time-warp rate change
- scene-exit finalization entry, before any fallback consumes stale data

Refresh should early-out when the vessel state digest has not meaningfully changed. A stable coasting vacuum orbit does not need a new cache every 5 seconds if the orbit, body, situation, throttle state, and terminal result are unchanged.

Actual Chain Only

Finalization must ignore KSP maneuver nodes. A maneuver node is an editor/planning UI object, not recorded trajectory data and not something the vessel will actually do unless the player later burns.

Implementation may use one of these strategies, in order of preference after code exploration:

1. Obtain a node-free patched-conic chain from stock APIs without mutating player nodes.
2. Temporarily suppress maneuver-node influence, refresh the solver, capture the real coast chain, and restore the player's solver/node state in `finally`.
3. Bypass stock patched-conic capture for node-contaminated chains and use the current orbit state plus Parsek's own conic/ballistic propagation.

The old `StoppedBeforeManeuver` behavior is not acceptable as the final design. It turned a hypothetical node into a real terminal boundary; Phase 2 replaces that with `EncounteredManeuverNode` detection and current-state propagation fallback.

Active Recorder Refresh

The active recorder can attempt the highest-fidelity cache:

- For live focused vessels, refresh from the live vessel state and stock node-free conic data when available.
- During powered flight or atmospheric flight, the cache is expected to change; refresh by cadence and forced triggers.
- During vacuum coast with no meaningful thrust, the cache should stabilize; repeated refreshes should be skipped unless an event invalidates the digest.

Background Recorder Refresh

Background recorders do not have the same solver access as the active vessel. They should still cache a reliable terminal outcome:

- Loaded background vessels in the physics bubble can refresh from live vessel/orbit state.
- On-rails background vessels can refresh from `vessel.orbit` and existing on-rails checkpoints.
- Stable vacuum orbits cache `Orbiting` plus terminal orbit metadata.
- Atmospheric or suborbital cases cache `Destroyed` at the predicted KSP deletion/destruction endpoint. If KSP deletes the vessel before the extrapolator's endpoint, the deletion event wins the terminal state but the cached tail still explains the end.

This does not create a cross-scene background recorder. The cache only protects the in-flight session that is already recording.

Applying a Cache

When a cache is consumed:

1. Find the last real authored UT on the `Recording` from `Points` , `TrackSections` , and non-predicted `OrbitSegments` .
2. Drop cached segments that end before that UT.
3. Clamp the first retained segment start to the last real UT when safe.
4. Append retained predicted segments with `isPredicted = true` .
5. Set `ExplicitEndUT` to the cache terminal UT if it extends the recording.
6. Set `TerminalStateValue` , terminal orbit/surface metadata, and terrain height when present.
7. Mark files dirty if the recording has already flushed sidcar data.
8. Emit a single summary log line naming the cache age, freshness, source, and consumer path.

Fresh live finalization has precedence over cache application. Cache application has precedence over trajectory-only inference. Trajectory-only inference remains the last-resort fallback.

Consumer Paths

The implementation must consume cache at these seams:

- scene exit for active and background recordings when the live finalizer declines or the vessel is missing
- manual commit or revert commit when a leaf/background vessel is already missing but has a usable cache
- active vessel destruction after crash coalescing decides the recording/tree should finalize
- background `OnBackgroundVesselWillDestroy`
- debris TTL, out-of-bubble, and missing-vessel termination in `BackgroundRecorder.CheckDebrisTTL`
- vessel-unloaded paths that remove or strand a background recording
- any future stop/abandon path that removes a recording from `BackgroundMap` or ends a live recorder without a fresh vessel

Focus switching is not itself a consumer unless the switch path actually ends the old recording. In the normal tree model, switching should transfer cache ownership to the background recorder instead.

Edge Cases

1. **Live scene exit with active vessel present.** Fresh finalization runs and wins. Cache is only a fallback.
2. **Scene exit after vessel object vanished.** Cache applies. If no cache exists, trajectory inference applies and logs degraded finalization.
3. **FlightGlobals unavailable in teardown/headless seam.** Cache applies without touching runtime-only APIs. If the cache is empty, existing guarded decline remains.
4. **Manual commit after a sibling already unloaded.** Cache applies before the non-scene-exit missing-vessel path stamps a weak destroyed fallback.
5. **Player has a planned maneuver node.** The node is ignored. The cached chain is the real coast/powered trajectory from current state, not the UI plan.
6. **Burn in progress.** Cache refreshes by cadence and throttle/engine triggers. Scene exit attempts one fresh refresh before consuming anything.
7. **Thrust stops in vacuum.** Forced refresh captures the new coast. Later refreshes can early-out until SOI/warp/orbit changes.
8. **Atmospheric unpowered flight.** Cache refreshes while loaded; if KSP unloads/deletes the vessel, terminal state is `Destroyed` .
9. **Stable vacuum orbit.** Cache can settle as `Orbiting` ; no synthetic destruction or 50-year horizon spawn should be invented for ordinary stable orbits.
10. **Suborbital vacuum arc with periapsis in atmosphere.** Cache predicts `Destroyed` even if the vessel is currently above atmosphere.
11. **Non-atmospheric body impact.** Cache predicts ground impact using the same ballistic terrain/sea-level fallback policy as the existing extrapolator.

12. **Background on-rails vessel lacks patched-conic solver.** Use orbit-state and ballistic/conic extrapolation only.
13. **SOI transition between refreshes.** Last cache may be stale; SOI event forces a refresh. If the vessel disappears in the same frame, the previous cache is still better than no tail and logs as stale.
14. **Crash coalescer still pending.** Do not apply cache until the coalescer has decided which tree leaves are terminal. The cache supplies the endpoint, not the structural BranchPoint decision.
15. **Debris TTL expiry while vessel still exists.** Consume cache if it is fresher than the TTL cutoff; otherwise refresh once from the live debris vessel before ending the recording.
16. **KSP deletes an unfocused atmospheric vessel earlier than Parsek's ballistic impact.** Stock outcome wins: terminal state `Destroyed`. Terminal UT is the earlier of deletion UT and predicted impact UT, and the cached tail is trimmed to that terminal UT. If no predicted impact UT exists, deletion UT is the terminal UT.
17. **Cache older than allowed freshness window.** Apply only with a `Stale` log if no live refresh is possible; otherwise refresh first.
18. **Recording has no real samples.** Do not append a free-floating tail. Mark degraded/destroyed according to the existing zero-point/debris policy.
19. **Cache terminal UT is before the last real sample.** Reject the cache and fall back to live/inference. This catches stale or mismatched ownership.
20. **Recording is already finalized.** Do not overwrite terminal metadata unless the consumer is explicitly repairing a known degraded finalization and logs the replacement.
21. **Hard quit without commit.** Cache is in-memory, so it is lost. This is acceptable: Parsek commits on recording end, not continuously through power loss.
22. **Old committed recordings.** No migration. They keep whatever terminal data they already have.
23. **Many background vessels.** Refresh is per recording, but digest early-outs and 5-second cadence keep cost bounded. Implementation should log aggregate refresh counts rather than per-vessel spam.

What Doesn't Change

- Actual sampled history is still authoritative. Synthetic tails never rewrite existing trajectory points.
- Recording sidecar format does not change for the cache. It is in-memory only.
- `OrbitSegment.isPredicted` remains the persisted marker for predicted/extrapolated segments that are applied to a committed recording.
- Committed recordings remain immutable after commit. Cache application happens before commit or as part of the existing finalization path.
- Background recording remains an in-flight, physics-bubble system. This design does not add cross-scene background simulation.
- Parsek does not simulate full atmospheric drag for unloaded vessels. It mirrors stock KSP's practical outcome for unfocused atmospheric deletion/destruction.
- Rewind to Separation UI and supersede semantics do not change here. They consume better terminal data after implementation.
- ERS/ELS access rules do not change. The cache is pre-commit finalization state and must not introduce new raw `RecordingStore.CommittedRecordings` or `Ledger.Actions` readers outside the existing grep-audit allowlist.

Backward Compatibility

There is no save migration and no new serialized cache node. Existing recordings keep their terminal state and predicted tail data as written.

If a future implementation discovers it needs a small persisted marker for "finalization was degraded", that must be a separate design update. The locked decision for this plan is in-memory cache only.

Diagnostic Logging

Use subsystem tags that make the finalization decision reconstructable from `KSP.Log` :

- `[FinalizerCache]` for refresh, cache status, cache application, cache rejection, and ownership transfer.
- `[Extrapolator]` for ballistic/conic propagation, terminal reason, limits, and failures.
- `[PatchedSnapshot]` for node-free stock chain capture and maneuver-node exclusion decisions.
- `[BgRecorder]` for background consume paths: destroy, unload, TTL, out-of-bubble, and persistence.
- `[Flight]` for tree-level finalization decisions and crash-coalescer timing.

Required log events:

- cache refresh attempt: recording id, vessel pid, owner, UT, reason, digest changed yes/no
- refresh decline: reason, whether existing cache remains usable
- cache stored: terminal state, terminal UT, segment count, source path, elapsed milliseconds
- ownership transfer: active -> background or background -> active with recording id and pid
- consumer entry: consumer path, live vessel found yes/no, cache status, cache age
- cache application summary: appended segment count, old/new `ExplicitEndUT` , terminal state
- cache rejection: stale, mismatched recording id/pid, terminal before last sample, empty payload, or already finalized
- degraded fallback: no live vessel and no usable cache, inference path used
- maneuver-node exclusion: node count or transition detected, selected node-free strategy, restore success/failure

Per-frame refresh checks must use rate-limited logs or aggregate summaries.

Test Plan

Every implementation PR must add tests in the same commit as the behavior it introduces.

Unit Tests

- `FinalizationCache_AppliesTailAfterLastRealSample` - fails if cached segments overwrite or duplicate authored trajectory history.
- `FinalizationCache_RejectsTerminalBeforeLastSample` - fails if stale cache can shorten a recording.
- `FinalizationCache_TransfersActiveToBackground` - fails if focus switching loses the cache for the old active recording.
- `FinalizationCache_StaleAppliesOnlyWhenNoLiveRefresh` - fails if stale cache silently wins over fresh live state.
- `FinalizationCache_InAtmosphereDeleteForcesDestroyed` - fails if stock atmospheric deletion can commit as `Orbiting` .
- `FinalizationCache_StableOrbitDoesNotRefreshEveryCadence` - fails if stable coast creates avoidable refresh churn.
- `PatchedSnapshot_IgnoresUiManeuverNodes` - fails if a planned node truncates finalization or becomes terminal data.
- `BackgroundCache_OnRailsOrbitingClassifiesOrbiting` - fails if stable background orbiters become destroyed/unfinished.
- `BackgroundCache_SuborbitalAtmosphericClassifiesDestroyed` - fails if falling boosters end as null/orbiting.
- `CacheApply_MarksFilesDirtyWhenSidecarAlreadyFlushed` - fails if background finalization data is lost across scene load.

Integration Tests

- Scene-exit finalization falls back to cache when `FlightGlobals` probe declines.
- Manual commit after a background vessel unloads applies cache instead of a generic missing-vessel fallback.
- Active crash path consumes cache after crash coalescer completion.
- Background `OnBackgroundVesselWillDestroy` consumes cache and persists the updated sidecar.
- Debris TTL out-of-bubble path consumes cache before removing `BackgroundMap`.
- Focus switch active -> background -> unload keeps one recording id and one cache.

Log Assertion Tests

- Refresh/store summary includes owner, recording id, terminal state, terminal UT, and reason.
- Cache rejection logs the exact rejection reason.
- Degraded fallback logs when no live vessel and no cache exist.
- Maneuver-node exclusion logs the node-free strategy used.
- Background consume paths log aggregate counts instead of one noisy line per frame.

In-Game Runtime Tests

- Booster separation in atmosphere: stage a controllable booster, fly away, let it unload/delete, commit, verify the booster recording has `Destroyed`, terminal UT beyond the last live sample, and a predicted tail.
- Mun orbiter sibling: undock lander/orbiter, land the active lander, commit, verify the orbiter finalizes as `Orbiting`.
- Scene exit mid-burn: exit during thrust, return to playback, verify ghost continues instead of freezing at the last sampled point.
- Planned maneuver node ignored: create a maneuver node, do not burn, finalize, verify the committed tail follows real coast and does not stop at the node.
- Focused crash: crash a focused vessel, commit through the destruction dialog path, verify the recording has a destroyed terminal and cache-extended endpoint.

Implementation Notes

The implementation plan lives in [dev/plans/recording-finalization-reliability.md](#). The high-level PR shape should be:

1. Cache primitives and pure apply/reject logic.
2. Active/background refresh producers, including node-free capture and maneuver-node correctness.
3. Scene-exit fallback consumption.
4. Premature-end consumers for unload, destroy, TTL, and crash paths.
5. Runtime coverage and calibration.

Each phase should be independently testable and reviewed against this document.

References

- [dev/plans/incomplete-ballistic-extrapolation.md](#) - shipped scene-exit predicted-tail plan and current extrapolator context.
- [dev/development-workflow.md](#) - design and plan/build/review workflow.
- [kOS Vessel Load Distance docs](#) - describes loaded/unloaded and packed/on-rails behavior in KSP.
- [KSP forum discussion: 25km auto delete](#) - community documentation of stock atmospheric auto-delete rules.
- [KSP forum discussion: unloaded ships do not aerobreak](#) - community discussion of unloaded vessels not receiving normal aerobraking.

- [KSP Wiki: Atmosphere](#) - body atmosphere overview and drag context.